

Provider.tf :

The screenshot shows a VS Code editor with the Explorer pane on the left displaying the file structure of a Terraform project. The file `provider.tf` is selected. The main editor shows the content of `provider.tf` with the following code:

```
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~>6.0"
6     }
7   }
8 }
9 provider "aws" {
10   region = var.aws_region
11   profile = "default"
12 }
```

The terminal window at the bottom shows the command `$ terraform plan` being executed, which results in an error:

```
Error: failed to get shared config profile, TERRAFORM

with provider["registry.terraform.io/hashicorp/aws"],
on provider.tf line 9, in provider "aws":
 9: provider "aws" {
```

S3 bucket creation :

The screenshot shows the AWS Management Console interface. The left-hand navigation pane is expanded to show the 'Amazon S3' service. The 'Buckets' section is selected, and the 'General purpose buckets' tab is active. A table lists the existing buckets, including `terraform-state-bucket1-21052002` in the `ap-south-1` region. The 'Create bucket' button is visible in the top right corner of the bucket list area.

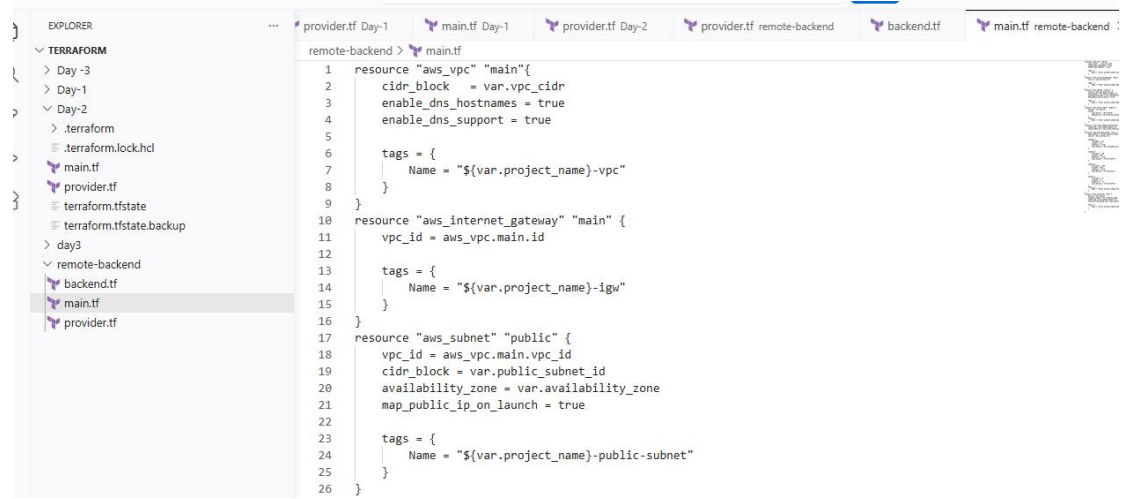
Backend.tf :

The screenshot shows a VS Code editor with the Explorer pane on the left displaying the file structure. The file `backend.tf` is selected. The main editor shows the content of `backend.tf` with the following code:

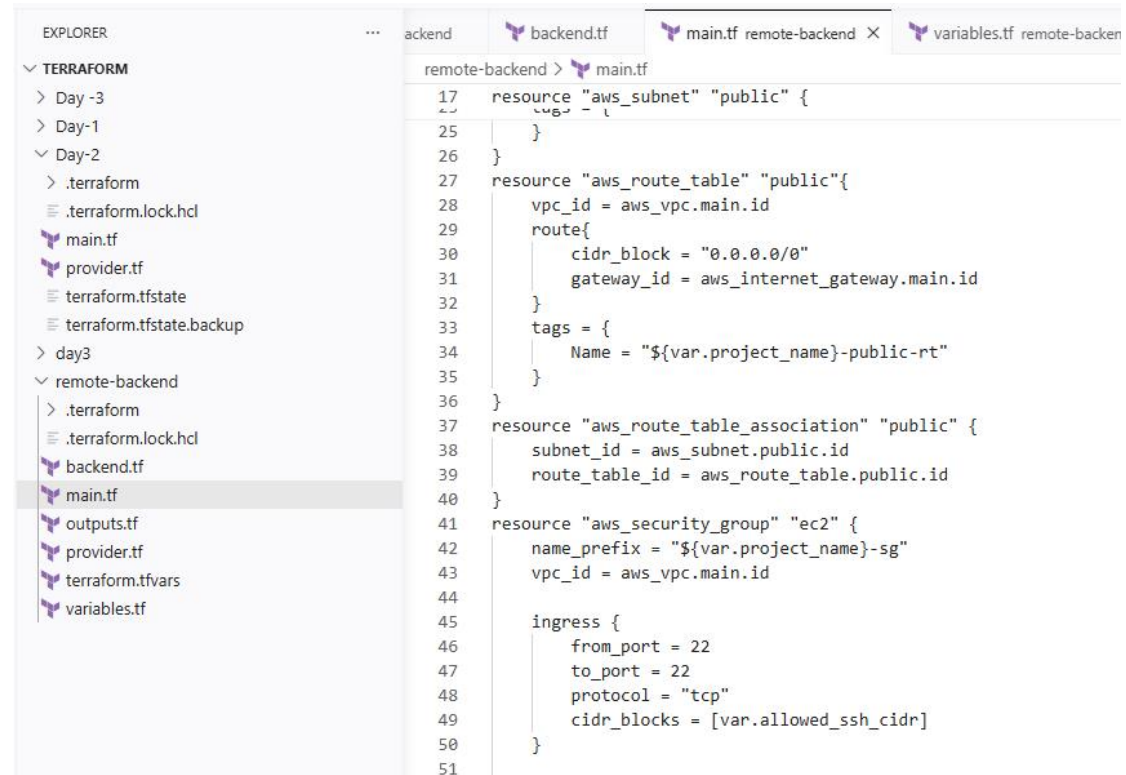
```
1 terraform {
2   backend "s3" {
3     bucket = "terraform-state-bucket1-21052002"
4     key    = "remote-backend/terraform.tfstate"
5     region = "ap-south-1"
6     encrypt = "true"
7     use_lockfile = "true"
8   }
9 }
```

The terminal window at the bottom shows the command `PS C:\Users\hp\gitrepos\Terraform>` being executed.

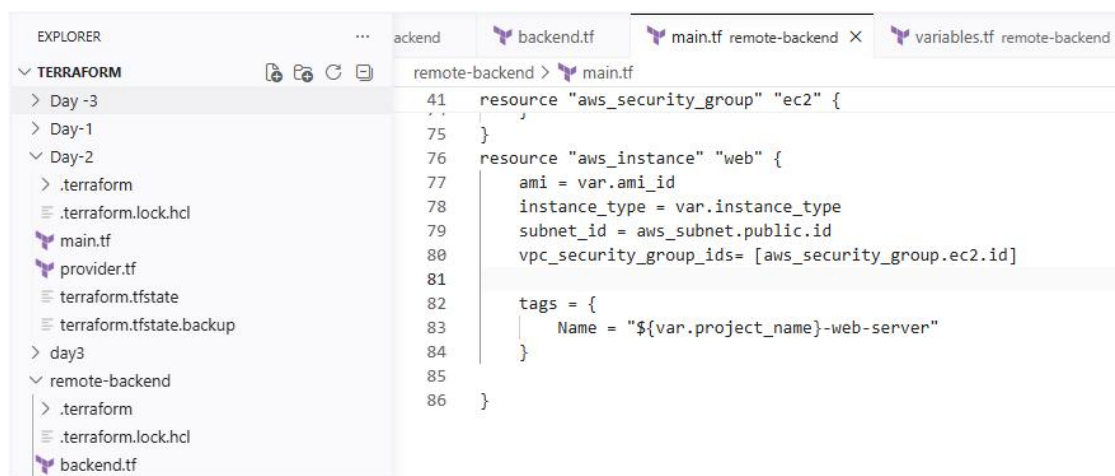
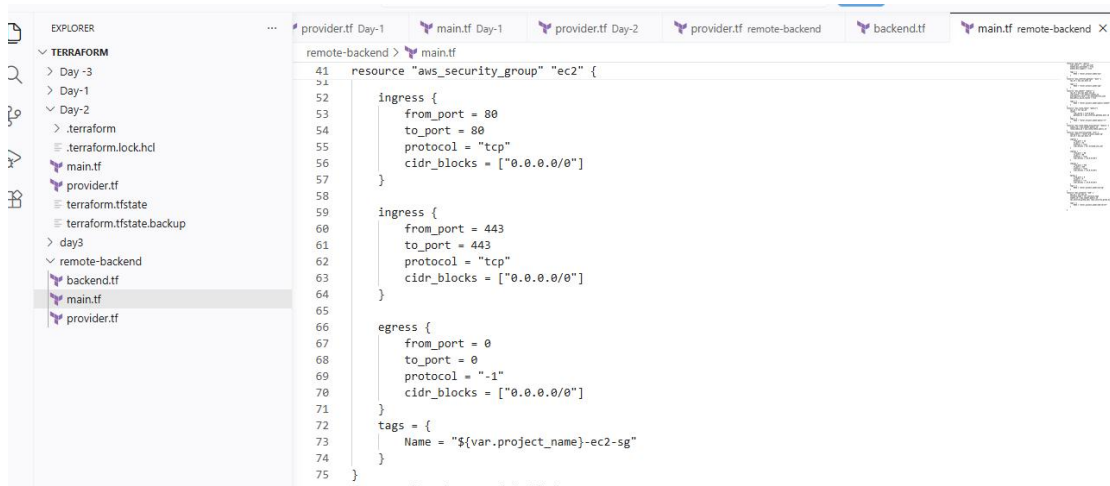
main.tf :



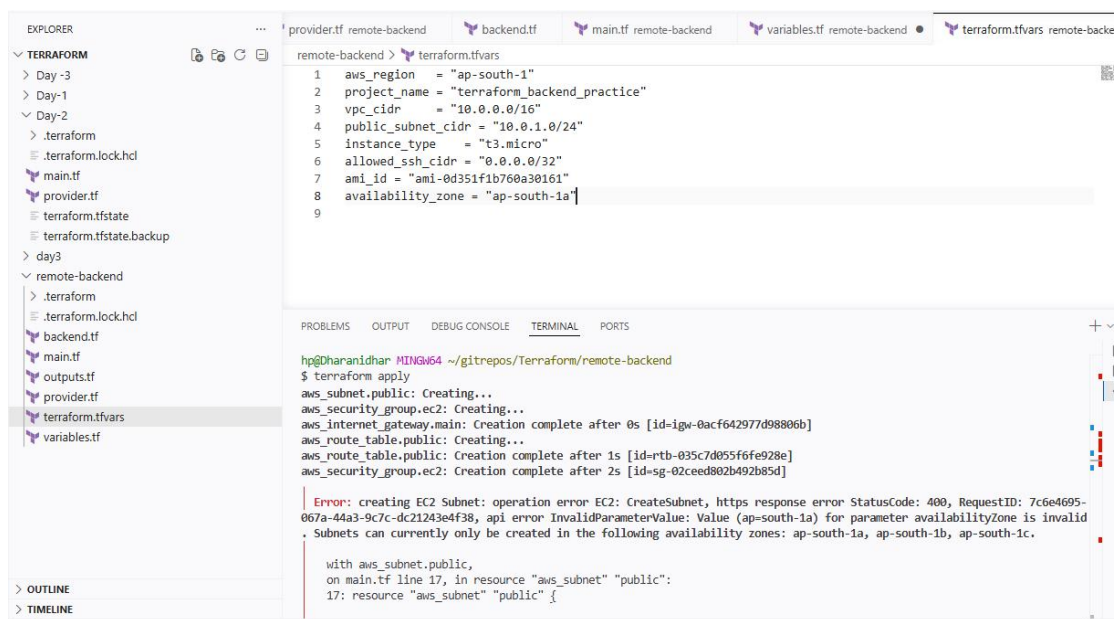
```
1 resource "aws_vpc" "main" {
2   cidr_block = var.vpc_cidr
3   enable_dns_hostnames = true
4   enable_dns_support = true
5
6   tags = {
7     Name = "${var.project_name}-vpc"
8   }
9 }
10 resource "aws_internet_gateway" "main" {
11   vpc_id = aws_vpc.main.id
12
13   tags = {
14     Name = "${var.project_name}-igw"
15   }
16 }
17 resource "aws_subnet" "public" {
18   vpc_id = aws_vpc.main.id
19   cidr_block = var.public_subnet_id
20   availability_zone = var.availability_zone
21   map_public_ip_on_launch = true
22
23   tags = {
24     Name = "${var.project_name}-public-subnet"
25   }
26 }
```

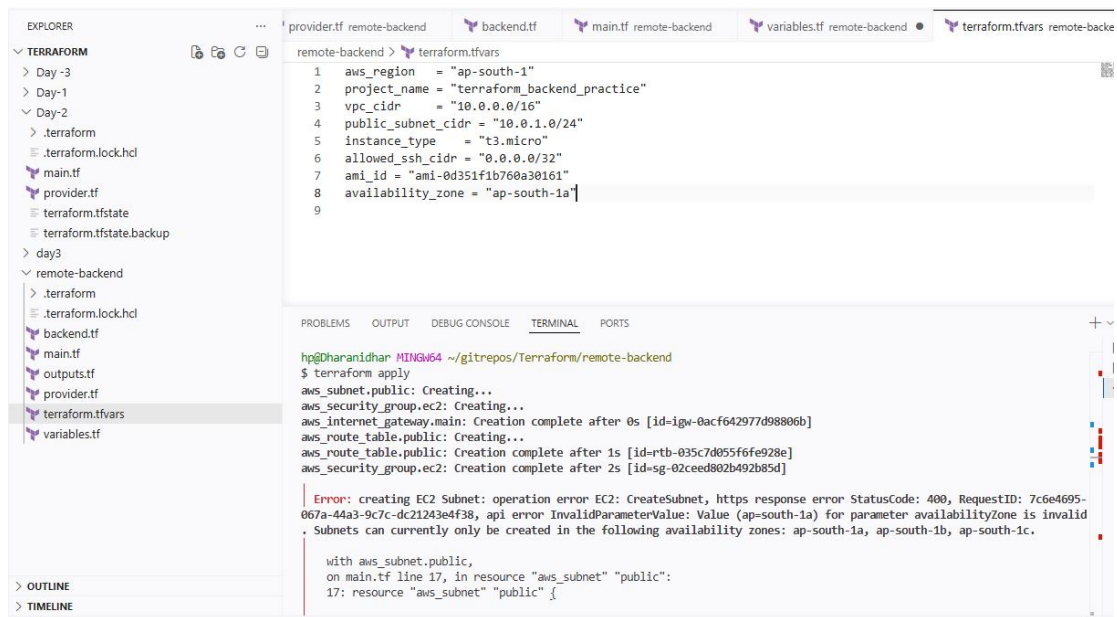


```
17 resource "aws_subnet" "public" {
25   }
26 }
27 resource "aws_route_table" "public" {
28   vpc_id = aws_vpc.main.id
29   route {
30     cidr_block = "0.0.0.0/0"
31     gateway_id = aws_internet_gateway.main.id
32   }
33   tags = {
34     Name = "${var.project_name}-public-rt"
35   }
36 }
37 resource "aws_route_table_association" "public" {
38   subnet_id = aws_subnet.public.id
39   route_table_id = aws_route_table.public.id
40 }
41 resource "aws_security_group" "ec2" {
42   name_prefix = "${var.project_name}-sg"
43   vpc_id = aws_vpc.main.id
44
45   ingress {
46     from_port = 22
47     to_port = 22
48     protocol = "tcp"
49     cidr_blocks = [var.allowed_ssh_cidr]
50   }
51 }
```

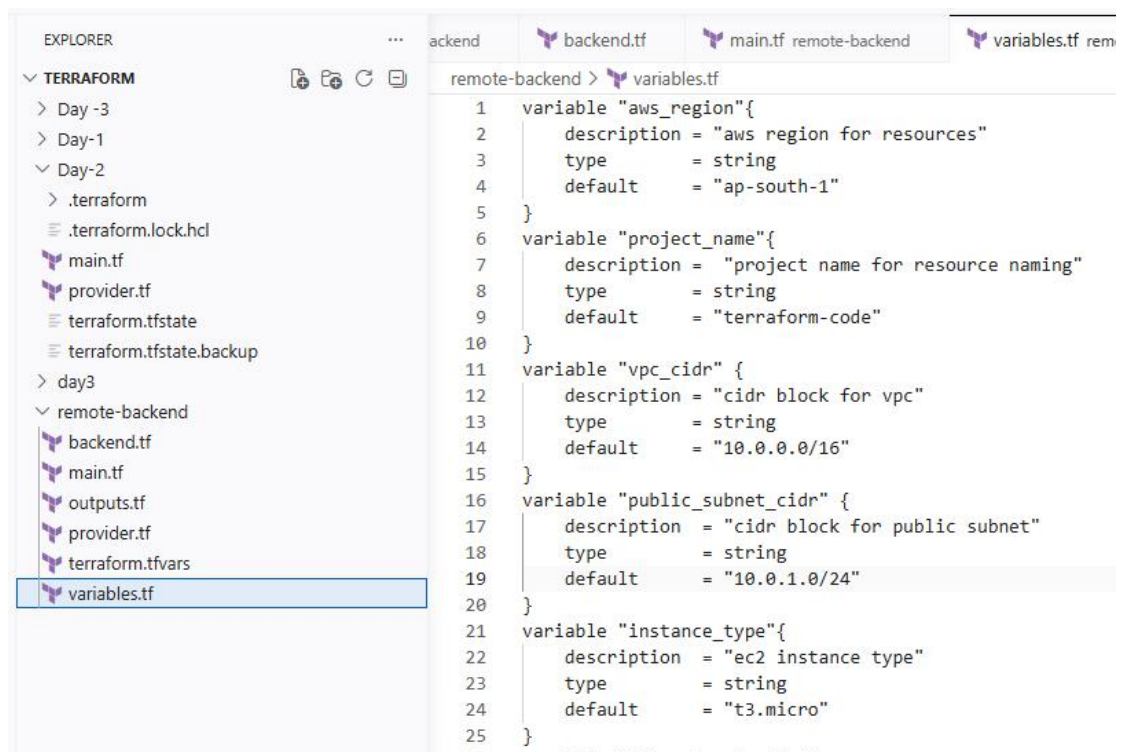


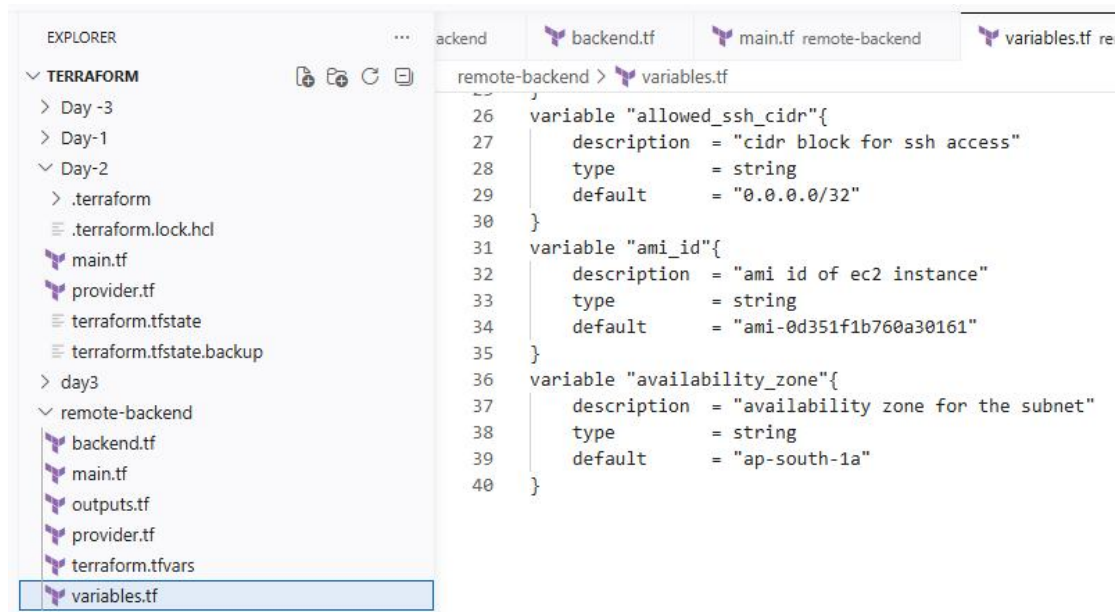
Terraform.tfvars :



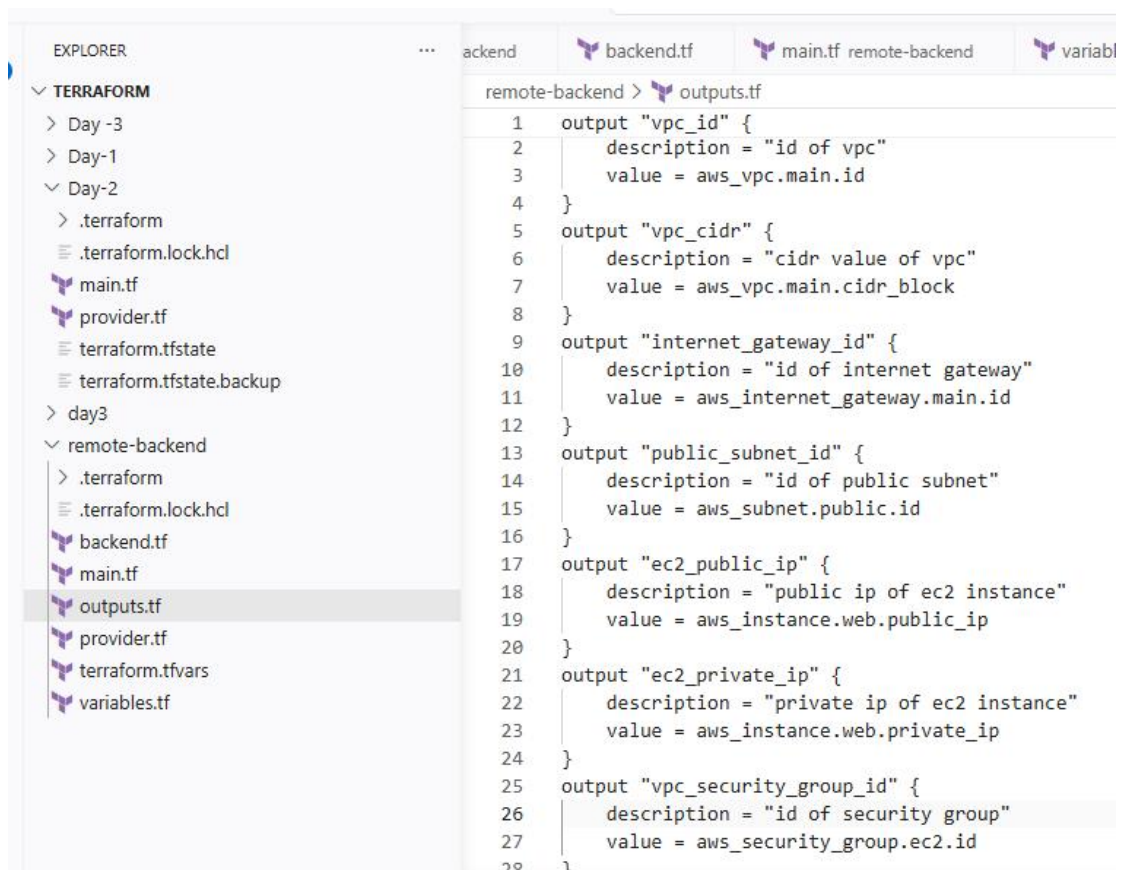


Variables.tf :





Outputs.tf :



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform
$ cd remote-backend

hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
$ aws s3 ls
2026-07-07 13:18:15 terraform-state-bucket1-21052002
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
$ terraform init
```

Error: Invalid default value for variable

on variables.tf line 19, in variable "public_subnet_cidr":
19: default = ["10.0.1.0/24"]

This default value is not compatible with the variable's type constraint: string required, but have tuple.

Error: Invalid default value for variable

on variables.tf line 29, in variable "allowed_ssh_cidr":
29: default = ["0.0.0.0/32"]
30: }
31: variable "ami_id"{
32: description = "ami id of ec2 instance"
33: type = string
34: default = "ami-0d351f1b760a30161"
35: }
36: variable "availability_zone"{

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
$ terraform init
```

Error: Extraneous label for output

on outputs.tf line 17:
17: output ec2_public_ip{
18: description = "public ip of ec2 instance"

Only 1 labels (name) are expected for output blocks.

Error: Invalid multi-line string

on outputs.tf line 17:
17: output ec2_public_ip{
18: description = "public ip of ec2 instance"

Quoted strings may not be split over multiple lines. To produce a multi-line string, either use the \n escape to represent a newline character or use the "heredoc" multi-line template syntax.

Error: Invalid string literal

on outputs.tf line 17:
17: output ec2_public_ip{
18: description = "public ip of ec2 instance"

This item is not valid in a string literal.

Error: Invalid multi-line string

on outputs.tf line 18:
18: description = "public ip of ec2 instance"
19: value = aws_instance.web.public_ip

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
$ terraform init
```

● Initializing provider plugins found in the configuration...

- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.53.0...
- Installed hashicorp/aws v6.53.0 (signed by HashiCorp)

Initializing the backend...

Successfully configured the backend "s3"! Terraform will automatically use this backend unless the backend configuration changes.

Terraform has created a lock file `.terraform.lock.hcl` to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
$ terraform plan
```

```
on main.tf line 49, in resource "aws_security_group" "ec2":
49:     cidr_block = var.allowed_ssh_cidr
```

An argument named "cidr_block" is not expected here. Did you mean "cidr_blocks"?

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
```

● \$ terraform plan

Error: Incorrect attribute value type

```
on main.tf line 49, in resource "aws_security_group" "ec2":
49:     cidr_blocks = var.allowed_ssh_cidr
    |
    | var.allowed_ssh_cidr is a string
```

Inappropriate value for attribute "cidr_blocks": list of string required, but have string.

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
```

● \$ terraform plan

Error: "var.allowed_ssh_cidr" is not a valid CIDR block: invalid CIDR address: var.allowed_ssh_cidr

```
with aws_security_group.ec2,
on main.tf line 41, in resource "aws_security_group" "ec2":
41: resource "aws_security_group" "ec2" {
```

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
● $ terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                    = "ami-0d351f1b760a30161"
  + arn                   = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone      = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized          = (known after apply)
  + enable_primary_ipv6    = (known after apply)
  + force_destroy          = false
  + get_password_data      = false
  + host_id                = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
  + id                     = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle     = (known after apply)
  + instance_state         = (known after apply)
  + instance_type          = "t3.micro"
  + ipv6_address_count     = (known after apply)
  + ipv6_addresses         = (known after apply)
+ ec2_private_ip          = (known after apply)
+ ec2_public_ip           = (known after apply)
+ internet_gateway_id     = (known after apply)
+ public_subnet_id        = (known after apply)
+ vpc_cidr                = "10.0.0.0/16"
+ vpc_id                  = (known after apply)
+ vpc_security_group_id   = (known after apply)
```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
○ $ terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                    = "ami-0d351f1b760a30161"
  + arn                   = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone      = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized          = (known after apply)
  + enable_primary_ipv6    = (known after apply)
  + force_destroy          = false
  + get_password_data      = false
```

```

hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
$ terraform apply
Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_private_ip      = (known after apply)
+ ec2_public_ip       = (known after apply)
+ internet_gateway_id = (known after apply)
+ public_subnet_id    = (known after apply)
+ vpc_cidr            = "10.0.0.0/16"
+ vpc_id              = (known after apply)
+ vpc_security_group_id = (known after apply)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_vpc.main: Creating...
aws_vpc.main: Creation complete after 2s [id=vpc-06240b645ed899240]
aws_internet_gateway.main: Creating...
aws_subnet.public: Creating...
aws_security_group.ec2: Creating...
aws_internet_gateway.main: Creation complete after 0s [id=igw-0acf642977d98806b]
aws_route_table.public: Creating...
aws_route_table.public: Creation complete after 1s [id=rtb-035c7d055f6fe928e]
aws_security_group.ec2: Creation complete after 2s [id=sg-02ceed802b492b85d]

Error: creating EC2 Subnet: operation error EC2: CreateSubnet, https response error StatusCode: 400, RequestID: 7c6e4695-067a-44a3-9c7c-dc21243e4f38, api error InvalidParameterValue: Value (ap=south-1a) for parameter availabilityZone is invalid . Subnets can currently only be created in the following availability zones: ap-south-1a, ap-south-1b, ap-south-1c.

Error: creating EC2 Subnet: operation error EC2: CreateSubnet, https response error StatusCode: 400, RequestID: 7c6e4695-067a-44a3-9c7c-dc21243e4f38, api error InvalidParameterValue: Value (ap=south-1a) for parameter availabilityZone is invalid . Subnets can currently only be created in the following availability zones: ap-south-1a, ap-south-1b, ap-south-1c.

with aws_subnet.public,
on main.tf line 17, in resource "aws_subnet" "public":
17: resource "aws_subnet" "public" {

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
$ terraform apply
aws_instance.web: Creation complete after 12s [id=i-0463e677cff4c13db]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

ec2_private_ip = "10.0.1.119"
ec2_public_ip  = "3.110.159.163"
internet_gateway_id = "igw-0acf642977d98806b"
public_subnet_id = "subnet-0f910de5c0b4d2e98"
vpc_cidr      = "10.0.0.0/16"
vpc_id        = "vpc-06240b645ed899240"
vpc_security_group_id = "sg-02ceed802b492b85d"

hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend

```

IAM
S3
EC2
VPC

Amazon S3
> Buckets
> terraform-state-bucket1-21052002
> remote-backend/

Amazon S3

Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems
Access management and security
Access Points
Access Points for FSx
Access Grants

Objects
Properties

Objects (1)

Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to [explicitly](#) grant them permissions. [Learn more](#)

☐	Name	Type	Last modified	Size	Storage class
☐	terraform.tfstate	tfstate	July 8, 2026, 00:29:38 (UTC+05:30)	17.3 KB	Standard

VPC dashboard < Your VPCs

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Data tables

Your VPCs (1/2) Info

Find VPCs by attribute or tag

	Name	VPC ID	State	Encryption c...	Encryption control ...
☐	-	vpc-0f9eb15138390ecb2	Available	-	-
☒	terraform_backend_practice-vpc	vpc-06240b645ed899240	Available	-	-

Details Info

VPC ID vpc-06240b645ed899240

DNS resolution Enabled

Main network ACL acl-01879391f0ccz799a

IPv6 CIDR (Network border group) -

Encryption control ID -

State Available

Tenancy default

Default VPC No

Network Address Usage metrics Disabled

Encryption control mode -

Block Public Access Off

DHCP option set dhcp-01268cea9e981531

IPv4 CIDR 10.0.0.0/16

Route 53 Resolver DNS Firewall rule groups -

DNS hostnames Enabled

Main route table rtb-0c8e5d1bd00e744cd

IPv6 pool -

Owner ID 354089337751

Resource map Info

Subnets (1)

Subnets within this VPC

ap-south-1a

terraform_backend_practice-public-subnet

10.0.1.0/24

No IPv6

Route tables (2)

Route network traffic to resources

rtb-0c8e5d1bd00e744cd

terraform_backend_practice-public-rt

Network Connections (1)

Connections to other networks

terraform_backend_practice-igw

EC2 < Instances

Dashboard

AWS Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Instances (1/1) Info

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

Instance state = running

Clear filters

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DI
☒	terraform_backend_practice-web-server	i-0463e677cf4c13db	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-3-110-15

EC2 < VPC

Security Groups > sg-02ceed802b492b85d

sg-02ceed802b492b85d - terraform_backend_practice-sg5ba74ef91638ed8cbee0d00af9

Details

Security group name terraform_backend_practice-sg5ba74ef91638ed8cbee0d00af9

Security group ID sg-02ceed802b492b85d

Description Managed by Terraform

VPC ID vpc-06240b645ed899240

Owner 354089337751

Inbound rules count 3 Permission entries

Outbound rules count 1 Permission entry

Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Inbound rules (3)

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
-	sgr-021dd50e0e6df608	IPv4	HTTP	TCP	80	0.0.0.0/0
-	sgr-0e55ef7b05cbe5310	IPv4	SSH	TCP	22	0.0.0.0/32
-	sgr-0ded7d8389c0ce9de	IPv4	HTTPS	TCP	443	0.0.0.0/0

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
$ terraform console

> aws_instance.web.id
"i-0463e677cff4c13db"
> aws_instance.web.public_ip
"3.110.159.163"
> aws_instance.web.private_ip
"10.0.1.119"
> aws_security_group.ec2.id
"sg-02ceed802b492b85d"
> aws_vpc.main.id

Error: Reference to undeclared resource

on <console-input> line 1:
  (source code not available)

A managed resource "aws_vpc" "main" has not been declared in the root module.

> aws_vpc.main.id
"vpc-06240b645ed899240"
> █
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/remote-backend
$ terraform destroy
Plan: 0 to add, 0 to change, 7 to destroy.

Changes to Outputs:
- ec2_private_ip      = "10.0.1.119" -> null
- ec2_public_ip       = "3.110.159.163" -> null
- internet_gateway_id = "igw-0acf642977d98806b" -> null
- public_subnet_id    = "subnet-0f910de5c0b4d2e98" -> null
- vpc_cidr             = "10.0.0.0/16" -> null
- vpc_id              = "vpc-06240b645ed899240" -> null
- vpc_security_group_id = "sg-02ceed802b492b85d" -> null

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_route_table_association.public: Destroying... [id=rtbassoc-026ade1afb45c5ec6]
aws_instance.web: Destroying... [id=i-0463e677cff4c13db]
aws_route_table_association.public: Destruction complete after 0s
aws_route_table.public: Destroying... [id=rtb-035c7d055f6fe928e]
aws_route_table.public: Destruction complete after 0s
aws_internet_gateway.main: Destroying... [id=igw-0acf642977d98806b]
aws_instance.web: Still destroying... [id=i-0463e677cff4c13db, 00m10s elapsed]
█
```